

International Journal of Information and Computer Security

ISSN online: 1744-1773 - ISSN print: 1744-1765

<https://www.inderscience.com/ijics>

LBCDA: lightweight blockchain-assisted cross-domain authentication scheme with privacy protection for SWIM

Lizhe Zhang, Jiahao Li, Yiao Ma, Zhijun Wu, Ruiqi Li

DOI: [10.1504/IJICS.2025.10071966](https://doi.org/10.1504/IJICS.2025.10071966)

Article History:

Received:	09 July 2024
Last revised:	09 April 2025
Accepted:	30 April 2025
Published online:	05 September 2025

LBCDA: lightweight blockchain-assisted cross-domain authentication scheme with privacy protection for SWIM

Lizhe Zhang

School of Safety Science and Engineering,
Civil Aviation University of China,
Tianjin 300300, China
Email: lzzhang@cauc.edu.cn

Jiahao Li

School of Electronic Information and Automation,
Civil Aviation University of China,
Tianjin 300300, China
Email: 2022022252@cauc.edu.cn

Yiao Ma, Zhijun Wu* and Ruiqi Li

School of Safety Science and Engineering,
Civil Aviation University of China,
Tianjin 300300, China
Email: 2023092118@cauc.edu.cn
Email: caucwu@263.net
Email: liruiqi0920@163.com
*Corresponding author

Abstract: As a global civil aviation information-sharing service platform, cross-domain authentication of system wide information management (SWIM) has attracted much attention. Due to the inability of existing authentication schemes to meet the needs of lightweight users such as aircraft for secure cross-domain access to SWIM in the aircraft access to SWIM application, we propose a lightweight blockchain-assisted cross-domain authentication scheme for SWIM. The scheme uses certificateless cryptography to generate user private keys and achieves mutual authentication and key agreement for cross-domain users based on the elliptic curve Diffie-Hellman. Additionally, cross-domain users share authentication public parameters and anonymous identity through a consortium blockchain, thereby protecting identity privacy and establishing trust among them. We use SVO logic to prove that the authentication protocol realises its design goals and utilise the formal verification tool Scyther to demonstrate the scheme's security. Compared to the four existing schemes, our scheme has lower computational and communication overhead.

Keywords: system wide information management; SWIM; blockchain; cross-domain authentication; lightweight.

Reference to this paper should be made as follows: Zhang, L., Li, J., Ma, Y., Wu, Z. and Li, R. (2025) ‘LBCDA: lightweight blockchain-assisted cross-domain authentication scheme with privacy protection for SWIM’, *Int. J. Information and Computer Security*, Vol. 28, No. 1, pp.20–50.

Biographical notes: Lizhe Zhang received his PhD in Safety Science and Engineering from the Civil Aviation University of China, Tianjin, China, in 2024. He is currently a Lecturer in the School of Safety Science and Engineering at the Civil Aviation University of China. His research interests include the security of air traffic management and system wide information management.

Jiahao Li received his Bachelor’s degree in Electronic Information Engineering from the Civil Aviation University of China, Tianjin, China, in 2022. He is currently pursuing his Masters degree in the School of Electronic Information and Automation at the Civil Aviation University of China. His research interests include civil aviation data security and privacy protection methods.

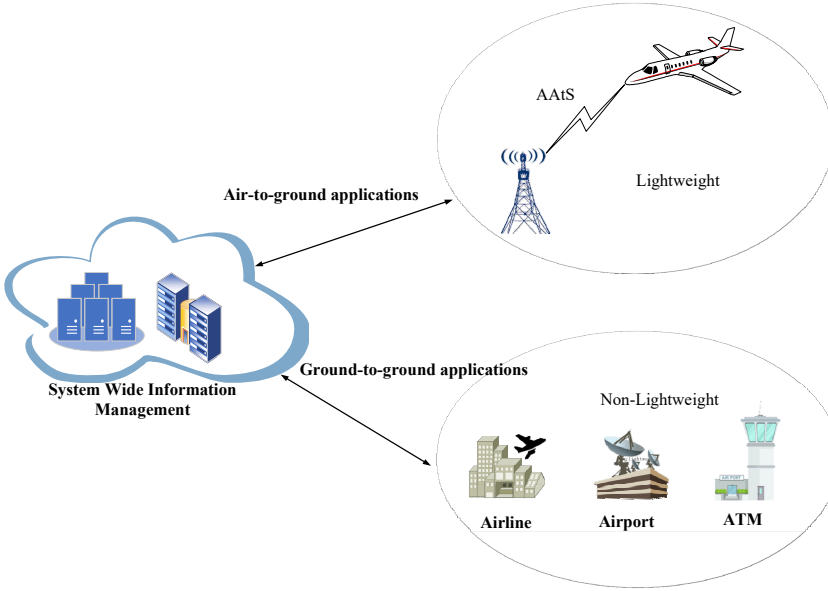
Yiao Ma received his Bachelor’s degree in Computer Science and Technology from the Civil Aviation University of China, Tianjin, China, in 2023. He is currently pursuing his Masters degree in the School of Safety Science and Engineering at the Civil Aviation University of China. His research interests include access control and privacy protection of large-scale information.

Zhijun Wu received his PhD in Cryptography from the Beijing University of Posts and Telecommunications, Beijing, China, in 2004. He is currently a Professor in the School of Safety Science and Engineering at the Civil Aviation University of China. His research interests include security of air traffic management and system wide information management.

Ruiqi Li received his PhD in Computer Science and Technology from the Nankai University, Tianjin, China, in 2021. He is currently a Lecturer in the School of Safety Science and Engineering at the Civil Aviation University of China. His research interests include fully homomorphic encryption, applied cryptography, and cloud computing security.

1 Introduction

System wide information management (SWIM) is a technology platform composed of software and hardware infrastructure that securely shares global air traffic management (ATM) information. To promote its development, the International Civil Aviation Organisation officially identified it as a key technology of the next-generation ATM system in 2005 (Crescenzo et al., 2010). SWIM adopts a service-oriented architecture and mainly provides four core functions: information transmission, interface management, enterprise service management, and security services (Qi and Lu, 2017). The SWIM application architecture is shown in Figure 1, which includes SWIM ground-to-ground and air-to-ground applications. Ground-to-ground applications are classified as non-lightweight applications, while air-to-ground applications are categorised as lightweight applications.

Figure 1 SWIM application architecture (see online version for colours)

The ground-to-ground applications of SWIM include sharing flight information and airspace conditions among air traffic control centres to achieve coordinated management of air traffic; sharing real-time flight status, ground service demands, and passenger information among airports and airlines to enhance operational efficiency; sharing resource allocation and personnel scheduling information among airport management departments and ground service providers to optimise ground service processes. Such applications have high resource demands and complex computations and are suitable for running on systems with stronger performance. The air-to-ground applications of SWIM connect aircraft through aircraft access to SWIM (AAtS) (Klein et al., 2015) to provide pilots with the latest weather, flight plans, Notices to Airmen, and other relevant aviation data. After collecting civil aviation business data, SWIM converts multi-source heterogeneous data into a unified standard using information exchange models such as flight information exchange model, aeronautical information exchange model, and weather information exchange model, providing standardised services for both ground-to-ground and air-to-ground users. SWIM system includes critical aviation business data such as flight plans and aeronautical information, facing security threats such as malicious attacks, system overload, and data leakage. If SWIM is attacked, it can lead to flight disorder, and in severe cases, it may endanger passengers' safety. Therefore, the SWIM system must ensure high security, with identity authentication serving as the crucial first step for secure access and playing a pivotal role in the system's secure operation.

SWIM adopts security assertion markup language (SAML) to achieve identity management, which may be subject to a centralised identity provider and face a single point of failure and the risk of identity privacy breaches. Moreover, for AAtS applications, due to the low communication bandwidth and data processing performance of aircraft, the authentication process may have insufficient performance. Additionally, air-to-ground communications transparently transmit data over open channels, making

aircraft vulnerable to replay attacks, spoofing attacks, man-in-the-middle attacks, and distributed denial of service (DDoS) attacks during the SWIM authentication process. To address these issues, this paper proposes a lightweight blockchain-assisted cross-domain authentication (LBCDA) scheme for SWIM. Using the blockchain, SWIM users can achieve decentralised cross-domain identity authentication to reduce redundant authentication overhead. Blockchain integration ensures the authenticity and trustworthiness of the identities during identity registration and mutual authentication processes among SWIM users. The main contributions of this paper are as follows:

- 1 A lightweight cross-domain identity authentication scheme for SWIM is proposed. LBCDA uses certificateless cryptography (CLC) algorithms to generate partial private keys by a central management authority (CA), which are then combined with the user's secret information to form complete private keys, effectively avoiding key escrow issues. Cross-domain authentication and key agreement (AKA) are implemented based on elliptic curve Diffie-Hellman (ECDH) algorithms, meeting the authentication needs of lightweight SWIM users such as aircraft.
- 2 LBCDA uploads system public parameters, users' anonymous identity, and public keys to a consortium blockchain named SWIM-Chain. It provides query services and identity privacy protection for all users. Trust relationships between cross-domain users are established through the decentralised, immutable, and traceable characteristics of SWIM-Chain.
- 3 We use SVO logic to demonstrate that the authentication protocol meets its intended goals, employs the Scyther to confirm its security, and utilises informal methods to analyse its security properties.
- 4 Experimental simulation comparison with four known schemes shows that the scheme has lower computational and communication overhead.

The remainder of this paper is organised as follows: Section 2 describes the related work, Section 3 provides an overview of the scheme, Section 4 describes the constructions of the scheme, Section 5 presents the logic verification and security analysis of the scheme, Section 6 conducts the performance analysis of the scheme, and Section 7 summarises the entire paper.

2 Related work

SWIM is a large distributed system, with subsystems possibly distributed in different regions involving ATMs, airlines, airports, etc. Civil aviation users in different regions need to access business resources from other regions through SWIM. Each domain is independently managed and authenticated, causing inconvenience for users who need to access data across domains. Currently, the security authentication of SWIM widely uses authentication mechanisms based on public key infrastructure (PKI) (Liu, 2020), which provide cross-domain authentication and are relatively easy to deploy. In the research on SWIM authentication architecture, Wu et al. (2014) improved the diameter/EAP-MD5 protocol to complete SWIM user identity authentication, and Zhang et al. (2023a) designed a SAML-PKI-LDAP-based unified identity authentication scheme for SWIM to achieve fast authentication of legitimate users. Blockchain technology is widely applied

in the civil aviation industry, with use cases including secure data sharing, aircraft maintenance, identity authentication, etc. The decentralisation and immutability of blockchain technology can promote secure information sharing, thereby improving security and aircraft airworthiness (Ahmad et al., 2021). Zhang et al. (2023b) developed a cross-domain authentication scheme based on consistent hash algorithms and blockchain for SWIM. This scheme reduces redundant identity authentication operations and addresses issues such as single points of failure and scalability challenges presented in traditional cross-domain authentication solutions. Lu et al. (2024) developed a blockchain-based trusted ATM security authentication protocol. As a crucial business system of the ATM, SWIM is adaptable to this authentication protocol. The above schemes are well suited for SWIM ground-to-ground applications, but they also have complex certificate management and key management problems, as well as high computational overhead and communication costs. The comparison of SWIM-related authentication schemes is shown in Table 1.

Table 1 Comparison of SWIM-related authentication schemes

<i>Schemes</i>	<i>Methodology</i>	<i>Mutual authentication</i>	<i>Identity privacy protection</i>	<i>Lightweight</i>	<i>Cross-domain</i>
Wu et al. (2014)	Diameter/EAP-MD5	√	×	×	√
Zhang et al. (2023a)	SAML-PKI-LDAP	√	×	×	√
Zhang et al. (2023b)	Blockchain + Consistent hashing	√	×	×	√
Lu et al. (2024)	T-ATMChain	√	×	×	√

Blockchain, as an emerging data management technology, has great potential to be applied in the authentication process. Traditional authentication schemes are primarily based on PKI (Wang et al., 2022) and identity-based cryptography (IBC) (Jia et al., 2020). Blockchain technology has been widely used in identity authentication research because of its characteristics of decentralisation and data immutability (Song et al., 2023). Integrating blockchain technology with PKI-based and IBC-based authentication protocols and utilising smart contracts (Zou et al., 2021) can establish trust relationships in untrusted environments, thereby enhancing the efficiency and security of cross-domain authentication.

In combining blockchain and PKI, Chen et al. (2022) developed a privacy-preserving cross-domain authentication scheme that integrates existing PKI and certificate transparency systems. The scheme uses blockchain and IPFS to decouple the control and storage layers, improving its performance and security. In combining blockchain and IBC, Shen et al. (2020) developed an efficient blockchain-assisted authentication scheme for the cross-domain IIOT environment, combining blockchain technology with identity-based signatures. This scheme achieves device anonymity and negotiates session keys, effectively addressing certificate management issues, though it still faces key escrow problems and cannot resist certain types of attacks. To address the key escrow issue, researchers have applied CLC (Al-Riyami and Paterson, 2003) to identity authentication. In this approach, the user’s private key is collaboratively generated by the key generation centre (KGC) and the authenticated user. Since the KGC cannot compute

the user's complete private key on its own, this method effectively protects the security of the private key.

Current blockchain-based cross-domain authentication solutions face critical challenges. Firstly, the processing capability of blockchain systems is limited, leading to potential slow response times when handling a large number of cross-domain authentication requests. Secondly, blockchain-based cross-domain authentication schemes may encounter complex certificate and key management issues, high communication overhead and computational cost. Additionally, they are susceptible to various network attacks, making it challenging to meet the demands of large-scale user cross-domain authentication.

To enhance the security and efficiency of cross-domain identity authentication while safeguarding user privacy, researchers have conducted a series of lightweight cross-domain authentication studies. Zhang et al. (2024b) designed a blockchain-based cross-domain authentication scheme for edge computing, where the authentication phase is based on digital signatures and digital certificates, and the key agreement phase uses a lightweight key agreement algorithm implemented with elliptic curve cryptography, effectively reducing computational and communication overhead. However, using digital certificates in the authentication phase inevitably increases the scheme's overhead. Dong et al. (2024) developed a lightweight blockchain-based certificate-free cross-domain authentication scheme for the industrial internet. This scheme generates user private keys through regional KGC for industrial enterprises and utilises smart contracts to query other enterprises, facilitating mutual authentication. Gong et al. (2023) designed a lightweight cross-domain mutual authentication (LCDMA) scheme suitable for mobile IoT. This scheme employs symmetric polynomials instead of the bilinear pairing to reduce computational overhead.

While meeting the lightweight requirements of cross-domain authentication, protecting the privacy of authentication entities is also crucial. Yang et al. (2022) and Cui et al. (2023) used pseudonym allocation mechanisms to protect identity privacy. Xue et al. (2022) used anonymity mechanisms and improved chameleon hashes to enhance security. Huang et al. (2022) used privacy protection technology to hide users' sensitive information on the chain, effectively protecting user identity privacy.

At present, most lightweight authentication scheme research focuses on the IoT environment (Yang et al., 2022; He et al., 2023; Liu et al., 2024; Zhang et al., 2024a). The SWIM network has characteristics similar to those of the IoT, and the characteristics of users, such as aircraft, are similar to those of sensors in the IoT. Therefore, the research on cross-domain authentication of IoT can provide a reference for SWIM authentication. Addressing the efficiency and privacy protection challenges SWIM air-to-ground application users face in identity authentication, this paper proposes a lightweight blockchain-assisted cross-domain secure authentication scheme for SWIM.

3 Overview of the proposed scheme

This section primarily describes the proposed scheme's system model, threat model, and design goals. The symbols used in the scheme and their descriptions are shown in Table 2.

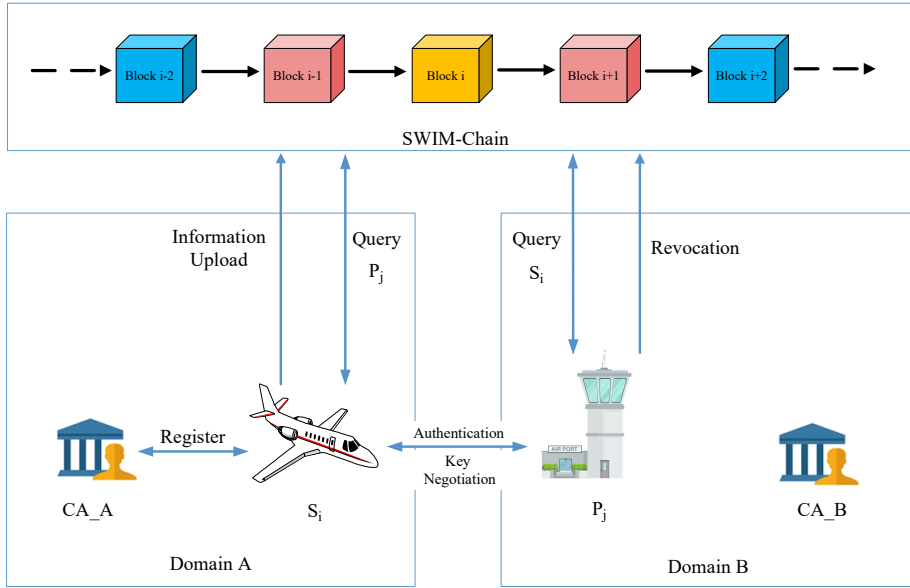
Table 2 The symbols and their descriptions

<i>Symbols</i>	<i>Descriptions</i>
U	User S_i/P_j
S_i	Data subscriber
P_j	Data publisher
CA	CA
ID	Unique identification of the user
AID	Anonymous ID of the user
Z_p^*	Set of non-zero integers less than p
G	Additive group of elliptic curves
g	A generator of G
SK_{CA}	Private key of CA
PK_{CA}	Public key of CA
d_U	Intermediate key generated by the CA for user U
SK_U	Private key of U
PK_U	Public key of U
x	Abscissa of a point on an elliptic curve
y	Ordinate of a point on an elliptic curve

3.1 System model

As shown in Figure 2, the proposed system model consists of four components: domain, SWIM users, CA, and SWIM-Chain. The details of the four components are as follows:

- 1 Domain: A domain refers to the range within which users in the same region trust each other, such as the ATM domain, airline domain, airport domain, etc. The ATM domain can be defined as domain A, and the airline domain can be defined as domain B. The trust scopes and relationships between different domains are independent of each other.
- 2 SWIM users (S_i/P_j): SWIM users can be either publishers or subscribers of SWIM data. The data publisher is denoted as P_j , and the data subscriber is denoted as S_i . P_j primarily collects civil aviation data, unifies heterogeneous data through data models, and publishes services to the SWIM registration centre. S_i can subscribe to service resources within its own domain and across domains according to its subscription permissions.
- 3 CA: the CA generates partial private keys for SWIM users. SWIM users combine the partial keys with their own generated secret information to create complete private keys, effectively avoiding key escrow issues.
- 4 SWIM-Chain: SWIM-Chain is a blockchain network built on the consortium blockchain Hyperledger Fabric, which assists the SWIM network in authentication tasks. In SWIM-Chain, smart contracts are used to store and query the anonymous ID of users and their public keys.

Figure 2 The proposed system model (see online version for colours)

Before starting the authentication, the user S_i needs to complete the registration with the CA of domain A and upload the identity information to SWIM-Chain. During the mutual authentication process, the authentication parties need to query each other's identity information first and then complete the authentication interaction and key negotiation. Registered users can revoke their identity information, such as anonymous IDs and public keys, which are published on the SWIM-Chain.

3.2 Threat model

This paper conducts a security analysis of the scheme using the Dolev-Yao security model (Dolev and Yao, 1983; Tang and Li, 2010). The capabilities of the attacker are defined as follows (Das and Namasudra, 2023):

- 1 The attacker can capture, modify, disrupt, redirect, delete, or replay any information sent through insecure channels.
- 2 The attacker is familiar with the interaction process of the designed protocol and can interfere with or participate in the execution of the protocol at any time.
- 3 The attacker cannot obtain any information sent through secure channels.

3.3 Design goal

The SWIM cross-domain authentication scheme should meet the following requirements:

- 1 Cross-domain authentication: when a SWIM subscriber needs to access data across domains, identity authentication with cross-domain users must be performed to ensure the legality of data access.

- 2 **Lightweight:** since SWIM data publishers and subscribers may involve resource-constrained devices such as aircraft with limited computing power and bandwidth, the resources and management tasks required to complete authentication should be minimised.
- 3 **Anonymity:** the anonymity of both local and cross-domain users must be ensured to prevent attackers from using identity information to carry out network attacks.
- 4 **Attack resistance:** the authentication scheme should resist common network attacks such as DDoS attacks, man-in-the-middle attacks, impersonation attacks, and replay attacks.

4 Construction

This section outlines the construction details of the scheme, encompassing four parts: initialisation phase, registration phase, cross-domain AKA phase, and authentication revocation phase.

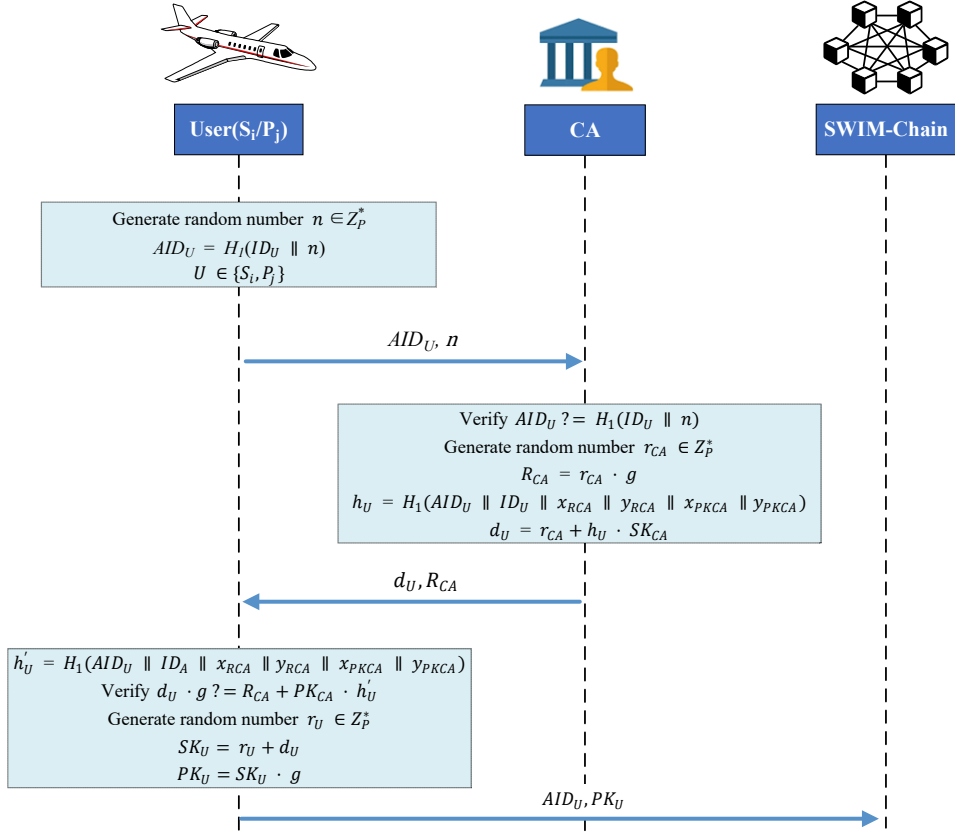
4.1 Initialisation phase

The data publishers and subscribers belong to each region according to the regional distribution and management structure. Each region achieves cross-domain authentication through SWIM-Chain. Within each region, a CA is responsible for registering users and assigning a unique identity identifier ID_U to all users in the domain. Users communicate with each other and with the CA through insecure channels. Users and CA communicate with SWIM-Chain through secure channels. First, the system selects one CA to complete steps 1 to 3 of the system initialisation and then proceeds to step 4. CAs from other domains synchronise the system's public parameters through SWIM-Chain and individually execute step 4 to generate their own private and public keys, then upload their CA identity information and public key to SWIM-Chain. The specific process for steps 1 to 4 is as follows:

- Step 1 Select an elliptic curve $E_q(a, b)$ defined over a prime field F_q , and choose an additive cyclic group G of order p on $E_q(a, b)$, where 0 is the order of F_q . g is the generator of G .
- Step 2 Select hash functions $H_1 : \{0, 1\}^* \rightarrow Z_p^*$; $H_2 : \{0, 1\}^* \times G \times G \rightarrow Z_p^*$; $H_3 : \{0, 1\}^* \times G \rightarrow Z_p^*$; $H_4 : \{0, 1\}^* \rightarrow \{0, 1\}^n$, where $n = 256$.
- Step 3 Write a smart contract to upload the system's public parameters $\{G, p, g, a, b, H_1, H_2, H_3, H_4\}$ to SWIM-Chain. The smart contract comprises three functions: data upload, data query, and data revocation. The data upload function is used to store identity information and public keys of entities such as data publishers, subscribers, and CAs. The data query function provides trusted data query services for entities in the system, such as querying the identity information and public keys published by S_i or P_j during the authentication phase. The data revocation function provides a service for revoking the registration information of system user entities.

Step 4 Each CA randomly generates $SK_{CA} \in Z_p^*$ as its private key and computes $PK_{CA} = SK_{CA} \cdot g$ as its public key. Each CA keeps SK_{CA} private and uploads PK_{CA} to SWIM-Chain.

Figure 3 Details of the registration phase (see online version for colours)



4.2 Registration phase

The primary tasks in this phase are to generate the user's anonymous ID, private key, and public key and to share the public key and anonymous ID through SWIM-Chain. Firstly, the SWIM user chooses a random number to complete the encapsulation and concealment of the real identity to generate the AID and sends the related identity information to the CA. Secondly, the CA verifies AID and generates the partial key for the registered user. After that, the CA sends the partial key to the registered user. Thirdly, the user generates private and public keys based on the partial key and publishes the public key and anonymous identity to the SWIM-Chain. The anonymous ID effectively protects the user's identity security. Users interact with the CA and SWIM-Chain. The CA assists in generating part of the private key, effectively alleviating the key escrow issue. Subsequently, the user's PK_U and the AID_U are sent to SWIM-Chain to achieve cross-domain sharing and mutual trust. The registration process is shown in Figure 3.

The implementation details are as follows:

- 1 $U \rightarrow CA: \{AID_U, n\}$. U generates a random number $n \in Z_p^*$ and calculates the anonymous identity AID_U by combining it with their real ID. Subsequently, the registration information $\{AID_U, n\}$ is sent to the CA.
- 2 $CA \rightarrow U: \{d_U, R_{CA}\}$. After receiving the registration information $\{AID_U, n\}$, the CA verifies the legality of AID_U to ensure message integrity. Once verified, the CA generates a random number r_{CA} and calculates R_{CA} . The CA then uses AID_U, ID_U, R_{CA} and PK_U to calculate h_U . Next, the CA generates U 's partial private key d_U to address the key escrow problem. Finally, the CA sends registration response $\{d_U, R_{CA}\}$ to U .
- 3 $U \rightarrow SWIM\text{-}Chain: \{AID_U, PK_U\}$. After receiving $\{d_U, R_{CA}\}$, U calculates the hash value h'_U . U then checks whether d_U and R_{CA} were modified during transmission. Once confirmed, U generates a random number r_U and computes his private key SK_U . Subsequently, U generates his public key PK_U based on SK_U . Finally, U sends identity information $\{AID_U, PK_U\}$ to SWIM-Chain through a secure channel for trusted storage.

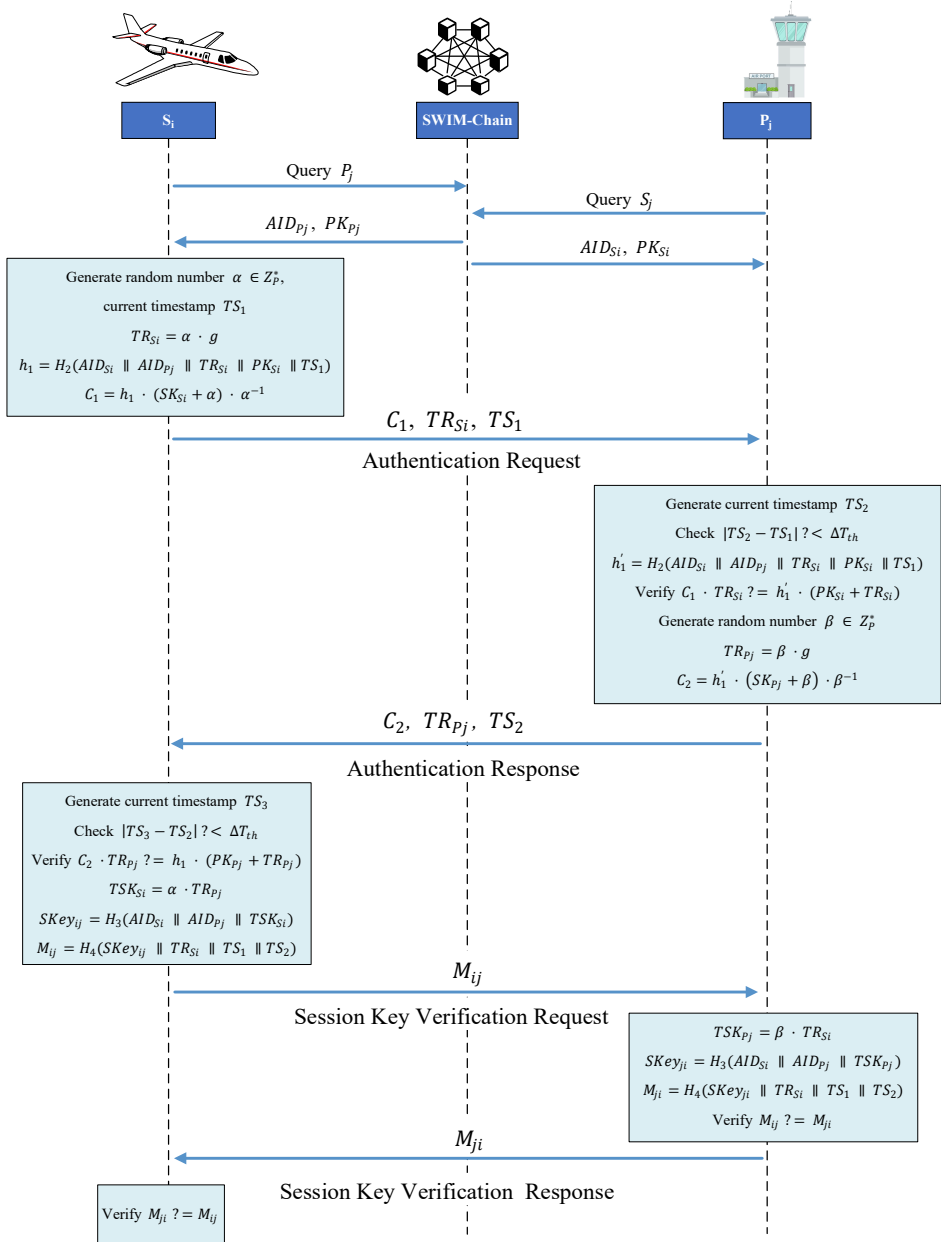
The construction of the registration phase can be found in Algorithm 1.

Algorithm 1 Registration

Input: n, ID_U

Output: AID_U, SK_U, PK_U

- 1 $U(S_i/P_j)$ generates $n \in Z_p^*$ and computes $AID_U = H_1(ID_U || n)$;
 - 2 $U(S_i/P_j)$ sends n and AID_U to CA;
 - 3 **if** U has been registered **then**
 - 4 return error;
 - 5 **else**
 - 6 CA verifies AID_U and computes $\{R_{CA}, h_U, d_U\}$;
 - 7 **end if**
 - 8 CA sends R_{CA} and d_U to U ;
 - 9 U computes $h'_U = H_1(AID_U || ID_A || x_{R_{CA}} || y_{R_{CA}} || x_{PK_{CA}} || y_{PK_{CA}})$;
 - 10 **if** $d_U \cdot g == R_{CA} + PK_{CA} \cdot h'_U$ **then**
 - 11 U computes $\{SK_U, PK_U\}$;
 - 12 **else**
 - 13 return error;
 - 14 **end if**
 - 15 U sends $\{AID_U, PK_U\}$ to SWIM-Chain;
-

Figure 4 Details of the cross-domain AKA phase (see online version for colours)

4.3 Cross-domain AKA phase

The cross-domain AKA process is shown in Figure 4, and the implementation details are as follows.

- 1 $U(S_i/P_j) \rightarrow \text{SWIM-Chain: } \{AID_{P_j}, PK_{P_j}, AID_{S_i}, PK_{S_i}\}$. S_i and P_j , respectively, query SWIM-Chain for each other's anonymous ID and public key to prepare for authentication. After completing the query, SWIM-Chain returns $\{AID_{P_j}, PK_{P_j}\}$ to S_i and $\{AID_{S_i}, PK_{S_i}\}$ to P_j .
- 2 $S_i \rightarrow P_j: \{C_1, TR_{S_i}, TS_1\}$. S_i generates a random number α and the current timestamp TS_1 . Subsequently, S_i calculates TR_{S_i} and h_1 using TR_{S_i} and h_1 . Then, S_i calculates C_1 . Finally, S_i sends the authentication request $\{C_1, TR_{S_i}, TS_1\}$ to P_j for mutual authentication.
- 3 $P_j \rightarrow S_i: \{C_2, TR_{P_j}, TS_2\}$. After receiving the authentication request, P_j obtains the current timestamp TS_2 and checks whether the communication has timed out. Then, P_j calculates h'_1 , and verifies if $C_1 \cdot TR_{S_i} ? = h'_1 \cdot (PK_{S_i} + TR_{S_i})$ to confirm the legitimacy of $\{C_1, TR_{S_i}, TS_1\}$. Next, P_j generates a random number β , calculates TR_{P_j} and C_2 . Finally, P_j sends the authentication response $\{C_2, TR_{P_j}, TS_2\}$ to S_i .
- 4 $S_i \rightarrow P_j: \{M_{ij}\}$. After receiving the authentication response, S_i obtains the current timestamp TS_3 and checks whether the communication has timed out. Then, S_i verifies if $C_2 \cdot TR_{P_j} ? = h_1 \cdot (PK_{P_j} + TR_{P_j})$ to confirm the integrity of the response message. After confirming the integrity, S_i calculates TSK_{S_i} and generates the session key $SKey_{ij}$. Then, S_i generates M_{ij} through a hashing operation. Finally, S_i sends the session key verification request $\{M_{ij}\}$ to P_j .
- 5 $P_j \rightarrow S_i: \{M_{ji}\}$. After receiving the session key verification request, P_j calculates TSK_{P_j} and generates the session key $SKey_{ji}$. Then, P_j performs a hashing operation based on $SKey_{ji}$ to generate M_{ji} and verifies the integrity of the session key. Finally, P_j sends the session key verification response $\{M_{ji}\}$ to S_i . After receiving $\{M_{ji}\}$, S_i verifies whether it equals M_{ij} . If they are equal, it confirms that the key agreement is completed.

Additionally, the construction of AKA Phase can be found in Algorithm 2.

Algorithm 2 Cross-domain AKA

Input: $AID_{P_j}, PK_{P_j}, AID_{S_i}, PK_{S_i}$

Output: $SKey_{ji}, SKey_{ij}$

- 1 S_i queries AID_{P_j} and PK_{P_j} from SWIM-Chain;
- 2 P_j queries AID_{S_i} and PK_{S_i} from SWIM-Chain;
- 3 S_i generates the current timestamp TS_1 and computes $\{TR_{S_i}, h_1, C_1\}$;
- 4 S_i sends authentication request $\{C_1, TR_{S_i}, TS_1\}$ to P_j ;
- 5 P_j generates the current timestamp TS_2 and checks whether the request is timeout;
- 6 **if** $|TS_2 - TS_1| < \Delta T_{th}$ **then**
- 7 P_j computes h'_1 and confirms the request;
- 8 P_j computes $\{TR_{P_j}, C_2\}$ and sends the authentication response $\{C_2, TR_{P_j}, TS_2\}$ to S_i ;
- 9 **else**
- 10 return error;
- 11 **end if**
- 12 S_i generates the current timestamp TS_3 and checks whether the response is timeout;

```

13  if  $|TS_3 - TS_2| < \Delta T_{th}$  then
14       $S_i$  confirms the response;
15       $S_i$  computes  $\{TSK_{S_i}, SKey_{ij}, M_{ij}\}$  and sends  $M_{ij}$  to  $P_j$ ;
16  else
17      return error;
18  end if
19   $P_j$  computes  $\{TSK_{P_j}, SKey_{ji}, M_{ji}\}$  and verifies the session key;
20  if  $M_{ij} = M_{ji}$  then
21       $P_j$  accepts the session key  $SKey_{ji}$  and send  $M_{ji}$  to  $S_i$ ;
22  else
23      return error;
24  end if
25   $S_i$  checks whether  $M_{ji}$  is equal to  $M_{ij}$ ;

```

4.4 Authentication revocation phase

When U (S_i/P_j) needs to revoke the registration information published on SWIM-Chain, U sends an identity revocation request $Rev_request$ to SWIM-Chain through a secure channel. SWIM-Chain verifies the user's identity and checks if it has been published. If verification is successful and the identity has been published, the revocation operation is executed, removing the user's registration information. The specific steps are as follows:

- Step 1 U generates the revocation request timestamp TS_{Rev} and constructs the revocation request message $Rev_request = \{AID, TS_{Rev}, Rev\}$, where Rev denotes the revocation operation identifier. U then signs $Rev_request$ using their private key SK_U to obtain signature message $Rev_request_sig$.
- Step 2 U embeds $Rev_request$, $Rev_request_sig$ and PK_U into the revocation request transaction, which triggers the revocation smart contract defined in SWIM-Chain.
- Step 3 SWIM-Chain checks if the public key information in the revocation request transaction is stored in the ledger. If the query is successful, this confirms that the user's identity information has been published.
- Step 4 SWIM-Chain uses the PK_U to verify the signature, ensuring that the revocation request originates from a legitimate user and preventing malicious users from forging revocation requests.
- Step 5 SWIM-Chain verifies whether the revocation request has timed out. If not, it triggers the smart contract to revoke the user's published AID_U and PK_U .

Once the identity is revoked, U must complete re-registration and republish their identity information on SWIM-Chain before accessing cross-domain data again.

The revocation of identity registration information is performed by Algorithm 3, where TS_{Rec} denotes the timestamp when the request is received, query and delete represent the query and delete functions in the smart contract, respectively, and $Qresult$ refers to the identity information returned by the query function.

Algorithm 3 Revocation

Input: $Rev_request, PK_U$
Output: success or error

- 1 $U(S_i/P_j)$ generates the request timestamp TS_{Rev} and constructs $Rev_request$;
- 2 U signs the $Rev_request$ and sends revocation request transaction to SWIM-Chain;
- 3 SWIM-Chain checks if the identity of U has been published: $Qresult \leftarrow Query(PK_U)$;
- 4 **if** $Qresult = null$ **then**
- 5 return error;
- 6 **end if**
- 7 SWIM-Chain verifies the $Rev_request_sig$;
- 8 SWIM-Chain checks whether the revocation request operation has timed out;
- 9 **if** $(TS_{Rec} - TS_{Rev}) < \Delta T_{th}$ **then**
- 10 SWIM
- 11 **else**
- 12 return error;
- 13 **end if**
- 14 return success;

5 Logic verification and security analysis

This section provides a comprehensive logic verification and security analysis of the scheme. First, SVO logic confirms that the scheme meets its intended authentication goals. Next, the Scyther verifies the scheme's security. Finally, an in-depth analysis and comparison of the scheme's security properties are conducted.

5.1 SVO logic verification

This paper uses SVO logic (Syverson and van Oorschot, 1996) to verify the authentication goals of our scheme. This method includes three main parts: authentication goals, assumptions, and proof, ensuring the theoretical completeness and correctness of the scheme.

5.1.1 Authentication goals

According to SVO logic, six authentication goals are defined for the proposed scheme (Dong et al., 2024).

- 1 Liveness confirmation: S_i believes P_j says M .
- 2 Entity authentication: S_i believes P_j says $F(M, N_a)$.
- 3 Secure key establishment: S_i believes $S_i \xleftarrow{SK_{ij}^-} P_j$.
- 4 Key confirmation: S_i believes $S_i \xleftarrow{SK_{ij}^+} P_j$.

- 5 Key freshness: S_i believes fresh ($SKey_{ij}$).
- 6 Mutual understanding of session key: S_i believes P_j says $P_j \xleftarrow{SKey_{ij}^-} S_i$.

In these authentication goals, M represents the message sent between the communicating parties, N_a represents the random number generated in the F algorithm, and $SKey_{ij}$ represents the shared key negotiated between two communicating parties. $SKey_{ij+}$ indicates that the $SKey_{ij}$ has been confirmed by both communicating parties as a shared key, while $SKey_{ij-}$ indicates that the key $SKey_{ij}$ has been sent by the sender but not yet confirmed by the receiver as a shared key.

5.1.2 Assumptions

- 1 Initialisation assumptions
 - AP₀: S_i believes fresh (TS_1), P_j believes fresh (TS_2).
 - AP₁: S_i believes fresh (α), P_j believes fresh (β).
 - AP₂: S_i believes $PK(P_j, PP_j)$, P_j believes $PK(S_i, PS_i)$.
- 2 Receive message assumptions
 - AP₃: S_i received (C_2, TR_{P_j}, TS_2), P_j received (C_1, TR_{S_i}, TS_1).
 - AP₄: S_i received M_{ji} , P_j received M_{ij} .
- 3 Intelligibility analysis of each received message (* represents a message that the subject cannot understand)
 - AP₅: S_i received ($*_1, *_2, TS_2$), P_j received ($*_3, *_4, TS_1$).
 - AP₆: S_i received $*_5$ P_j received $*_6$.
- 4 The subject's understanding of the message
 - AP₇: S_i believes (S_i received ($*_1, *_2, TS_2$) $\supset S_i$ received ($([\{h'_1\}_{(SK_{P_j}, \beta)}]_{\beta^{-1}}, \{g\}_{\beta}, TS_2)$)).
 - AP₈: P_j believes (P_j received ($*_3, *_4, TS_1$) $\supset P_j$ received ($([\{h'_1\}_{(SK_{S_i}, \alpha)}]_{\alpha^{-1}}, \{g\}_{\alpha}, TS_1)$)).
 - AP₉: S_i believes (S_i received $*_5 \supset S_i$ received ($H_4(SK_{y_{ji}} || TR_{S_i} || TS_1 || TS_2)$, $S_i \xleftarrow{SK_{y_{ji}}} P_j$, fresh ($SK_{y_{ji}}$))).
 - AP₁₀: P_j believes (P_j received $*_6 \supset P_j$ received ($H_4(SK_{y_{ji}} || TR_{S_i} || TS_1 || TS_2)$, $S_i \xleftarrow{SK_{y_{ij}}} P_j$, fresh ($SK_{y_{ij}}$))).

5.1.3 Proof

- Step 1 Based on assumptions AP₅, AP₇, SVO axiom A1, and the MP rule, the following conclusion can be derived:

Con₁: S_i believes S_i received ($([\{h'_1\}_{(SK_{P_j}, \beta)}]_{\beta^{-1}}, \{g\}_{\beta}, TS_2)$).

Step 2 Based on conclusion Con₁ and axiom A7, the following can be derived:

$$S_i \text{ believes } S_i \text{ received } ([\{h'_1\}_{(SK_{P_j}, \beta)}]_{\beta^{-1}}, TS_2) \supset h'_1, TS_2),$$

Furthermore, based on assumption AP₀ and axiom A17, the following can be derived:

$$S_i \text{ believes } S_i \text{ received } (\text{fresh}([\{h'_1\}_{(SK_{P_j}, \beta)}]_{\beta^{-1}}) \supset \text{fresh}(h'_1)),$$

Finally, based on axiom A1 and the MP rule, the following conclusion can be derived:

$$\text{Con}_2: S_i \text{ believes fresh } (h_1).$$

Step 3 Based on conclusion Con₁ and axiom A7, the following can be derived:

$$S_i \text{ believes } S_i \text{ received } [\{h'_1\}_{(SK_{P_j}, \beta)}]_{\beta^{-1}},$$

Based on assumption AP₂ and axiom A9, the following can be derived:

$$S_i \text{ believes } S_i \text{ received } \{h'_1\}_{(SK_{P_j}, \beta)},$$

Based on axiom A8, the following can be derived:

$$S_i \text{ believes } S_i \text{ received } h'_1,$$

Finally, based on assumption AP₂, axioms A4 and A1, and the MP rule, the following conclusion can be derived:

$$\text{Con}_3: S_i \text{ believes } P_j \text{ said } h'_1.$$

Step 4 Based on conclusions Con₂, Con₃, axioms A19 and A1, and the MP rule, the following conclusion can be derived:

$$\text{Con}_4: S_i \text{ believes } P_j \text{ says } h'_1.$$

Step 5 Based on assumptions AP₆ and AP₉, axiom A1, and the MP rule, the following conclusion can be derived:

$$\text{Con}_5: S_i \text{ believes } S_i \text{ received } (H_4(SK_{y_{ji}} || TR_{S_i} || TS_1 || TS_2), S_i \xleftarrow{SK_{y_{ji}}} P_j, \text{fresh}(SK_{y_{ji}})).$$

Step 6 Based on conclusion Con₅, and axiom A7, the following can be derived:

$$S_i \text{ believes } S_i \text{ received } H_4(SK_{y_{ji}} || TR_{S_i} || TS_1 || TS_2),$$

Based on assumption AP₀ and axiom A18, the following can be derived:

$$S_i \text{ believes } S_i \text{ received fresh } (H_4(SK_{y_{ji}} || TR_{S_i} || TS_1 || TS_2)),$$

Finally, based on axiom A1 and the MP rule, the following conclusion can be derived:

$$\text{Con}_6: S_i \text{ believes fresh } (H_4(SK_{y_{ji}} || TR_{S_i} || TS_1 || TS_2)).$$

Step 7 Based on assumption AP₂ and axiom A5, the following can be derived:

$$S_i \xleftarrow{SKey_{ji}} P_j,$$

Based on conclusion Con₁ and axiom A7, the following can be derived:

S_i believes S_i received $\{g\}_\beta$,

Based on axiom A10, the following can be derived:

S_i believes (S_i has $\{g\}_\beta$),

Finally, based on axiom A1, the MP rule, and the properties of non-confirmed shared keys, the following conclusion can be derived:

$$\text{Con}_7: S_i \text{ believes } S_i \xleftarrow{SKey_{ji}^-} P_j.$$

Step 8 Based on conclusion Con₅, axioms A7 and A3, the following can be derived:

S_i believes P_j said (P_j has $SKey_{ji}$),

Based on conclusion Con₆, axioms A18 and A19, and the MP rule, the following can be derived:

S_i believes P_j says (P_j has $SKey_{ji}$),

Finally, based on conclusion Con₇, axiom A1, the MP rule, and the properties of confirmed shared keys, the following conclusion can be derived:

$$\text{Con}_8: S_i \text{ believes } S_i \xleftarrow{SKey_{ji}^+} P_j.$$

Step 9 Based on conclusion Con₆, axioms A18 and A1, and the MP rule, the following conclusion can be derived:

Con₉: S_i believes fresh ($SKey_{ji}$).

Step 10 Based on conclusion Con₅ and axiom A7, the following can be derived:

$$S_i \text{ believes } S_i \text{ received } (H_4(SKey_{ji} || TR_{Si} || TS_1 || TS_2), S_i \xleftarrow{SKey_{ji}} P_j),$$

Based on axioms A3, A1, and the MP rule, the following conclusion can be derived:

$$\text{Con}_{10}: S_i \text{ believes } P_j \text{ said } H_4(SKey_{ji} || TR_{Si} || TS_1 || TS_2).$$

Step 11 Based on conclusions Con₆ and Con₁₀, axioms A19 and A1, and the MP rule, the following conclusion can be derived:

$$\text{Con}_{11}: S_i \text{ believes } P_j \text{ says } H_4(SKey_{ji} || TR_{Si} || TS_1 || TS_2).$$

Step 12 Based on conclusions Con₅ and Con₈, axioms A3 and A1, the MP rule, and the properties of non-confirmed shared keys, the following conclusion can be derived:

$$\text{Con}_{12}: S_i \text{ believes } P_j \text{ sees } P_j \xleftarrow{SKey_{ji}^-} S_i.$$

This section uses SVO logic to verify that the scheme successfully achieves the desired goals during the AKA phase. The SVO verification results are shown in Table 3. Conclusion Con₄ demonstrates the alive confirmation goal for P_j ; conclusion Con₁₁ demonstrates the entity authentication goal for P_j ; conclusion Con₇ demonstrates the secure key establishment between both parties; conclusion Con₈ demonstrates the key confirmation for both parties; conclusion Con₉ demonstrates the freshness of the session key for both communicating parties; conclusion Con₁₂ demonstrates that S_i believes that P_j has successfully obtained the relevant information and can compute the session key. Since the scheme is a symmetric protocol, through the related proofs by S_i regarding P_j , P_j can also derive the corresponding proofs regarding S_i , ensuring that both parties successfully complete the AKA process.

Table 3 SVO verification result

<i>Conclusion</i>	<i>Authentication goal</i>	<i>Verification result</i>
Con ₄	Liveness confirmation	√
Con ₁₁	Entity authentication	√
Con ₇	Secure key establishment	√
Con ₈	Key confirmation	√
Con ₉	Key freshness	√
Con ₁₂	Mutual understanding of session key	√

5.2 Formal analysis

In this section, we utilise the Scyther to verify the scheme's security. Users can evaluate whether the protocol meets the required security properties by declaring the desired properties. Scyther will display the attack process graphically if any attack is detected. Users can utilise the security protocol description language (SPDL) to implement the message interaction processes between roles in the protocol for a detailed security analysis (Mall et al., 2021). It is noteworthy that Scyther supports both bounded and unbounded protocol verification. For complex protocols, verification time grows exponentially with the number of running instances. Scyther cannot handle protocol verification involving algebraic properties. Moreover, its attack patterns are generated through a pattern refinement algorithm, providing a finite representation of behaviours. As a result, these patterns may not cover all possible attack scenarios. However, our scheme is a lightweight authentication protocol and Scyther can meet the need for formalised security analysis. The proposed LBCDA scheme can be verified to be secure within a finite number of running instances.

5.2.1 Registration phase analysis

The SPDL for the registration phase of the scheme is shown in Figure 5.

IDU represents the identity identifier of U. rCA represents the random number generated by the CA. sk(CA) represents the CA's private key. Scyther analyses the secrecy of IDU sent by U, the secrecy of rCA and sk(CA) sent by the CA, and the Alive, Weakagree, Niagree, and Nisynch properties of both roles.

The verification results of the registration phase are shown in Figure 6.

Figure 5 SPDL for verification in the registration phase

```

usertype Number,Point;
const g:Point;
const XPKCA,YPKCA,XRCA,YRCA:Number;
hashfunction H1;
protocol Register(U,CA)
{
  role U
  {
    fresh n; var IDU,rCA; var rca,dspi:Function;
    macro AIDU = H1(IDU,n);macro RCA = rca(rCA,g);
    macro hU = H1(AIDU,IDU,XRCA,YRCA,XPKCA,YPKCA);
    macro dU = dspi(rCA,{hU}sk(CA));
    recv_1(CA,U,{IDU,CA}pk(U)sk(CA));
    send_2(U,CA,AIDU,n);
    recv_3(CA,U,dU,RCA);
    claim(U,Secret,IDU); claim(U,Alive);
    claim(U,Weakagree); claim(U,Niagree);
    claim(U,Nisynch);
  }

  role CA
  {
    fresh IDU,rCA; fresh rca,du:Function; var n;
    macro AIDu = H1(IDU,n); macro RCA = rca(rCA,g);
    macro hU = H1(AIDU,IDU,XRCA,YRCA,XPKCA,YPKCA);
    macro dU = du(rCA,{hU}sk(CA));
    send_1(CA,U,{IDU,CA}pk(U)sk(CA));
    recv_2(U,CA,AIDU,n);
    send_3(CA,U,dU,RCA);
    claim(CA,Secret,rCA); claim(CA,Secret,sk(CA));
    claim(CA,Alive); claim(CA,Weakagree);
    claim(CA,Niagree); claim(CA,Nisynch);
  }
}

```

Figure 6 Verification results of the registration phase (see online version for colours)

Scyther results : verify

Claim				Status	Comments
Register	U	Register,U1	Secret IDU	Ok	No attacks within bounds.
		Register,U2	Alive	Ok	No attacks within bounds.
		Register,U3	Weakagree	Ok	No attacks within bounds.
		Register,U4	Niagree	Ok	No attacks within bounds.
		Register,U5	Nisynch	Ok	No attacks within bounds.
	CA	Register,CA1	Secret rCA	Ok	No attacks within bounds.
		Register,CA2	Secret sk(CA)	Ok	No attacks within bounds.
		Register,CA3	Alive	Ok	No attacks within bounds.
		Register,CA4	Weakagree	Ok	No attacks within bounds.
		Register,CA5	Niagree	Ok	No attacks within bounds.
Register,CA6	Nisynch	Ok	No attacks within bounds.		

Done.

The Scyther did not detect any methods that could be used for attacks. Specifically, the user identifier IDU of U, the random number rCA, and the sk(CA) were all proven to be

secure. Additionally, the verification results of the Alive, Weakagree, Niagree, and Nisynch properties for both entities were proven to be secure. Therefore, it can be confirmed that the scheme is secure in the registration phase.

5.2.2 Cross-domain AKA phase analysis

The SPDL for the AKA phase is shown in Figure 7. Alpha and Beta, respectively, denote the random numbers generated by S_i and P_j . Alpha' and Beta', respectively, represent the inverses of Alpha and Beta. $sk(S_i)$ and $sk(P_j)$ respectively denote the private keys of S_i and P_j . $SKey_{ij}$ and $SKey_{ji}$ represent the session keys negotiated between S_i and P_j . Scyther analyses the secrecy of the random number Alpha and the $sk(S_i)$ sent by S_i , the secrecy of the random number Beta and the $sk(P_j)$ sent by P_j , the confidentiality of the session keys $SKey_{ij}$ and $SKey_{ji}$, and the Alive, Weakagree, Niagree, and Nisynch properties of both roles.

Figure 7 SPDL for verification in the AKA phase

```

usertype Timestamp,String,Point;
const g: Point;
const AIDSi,AIDPj: String;
hashfunction H1,H2,H3,H4;
protocol SWIM(Si,Pj)
{
  role Si
  {
    fresh Alpha,Alpha';var Beta,Beta',SKeyij;
    fresh pk1,mul,add:Function;
    fresh TS1:Timestamp;var TS2:Timestamp;
    macro SKSi = sk(Si);
    macro SKPj = sk(Pj);
    macro PKSi = pk1(SKSi,g);
    macro PKPj = pk1(SKpj,g);
    macro TRsi = mul(Alpha,g);
    macro TRpj = mul(Beta,g);
    macro h1 = H2(AIDSi,AIDPj,TRsi,PKSi,TS1);
    macro h1' = H2(AIDSi,AIDPj,TRsi,PKSi,TS1);
    macro C1 = {(h1)add(SKSi,Alpha)}Alpha';
    macro C2 = {(h1')add(SKpj,Beta)}Beta';
    macro TSKSi = mul(Alpha,TRpj);
    macro TSKPj = mul(TRsi,Beta);
    macro SKeyS = H3(AIDSi,AIDPj,TSKSi);
    macro SKeyP = H3(AIDSi,AIDPj,TSKPj);
    macro Mij = H4(SKeyS,TRsi,TS1,TS2);
    macro Mji = H4(SKeyP,TRsi,TS2,TS1);
    send_1(Si,Pj,C1,TRsi,TS1);
    recv_2(Pj,Si,C2,TRpj,TS2);
    send_3(Si,Pj,Mij);
    recv_4(Pj,Si,Mji);
    match (SKeyij,SKeyS);
    claim(Si,Secret,Alpha);
    claim(Si,Secret,SKSi);
    claim(Si,SKR,SKeyij);
    claim(Si,Alive);
    claim(Si,Weakagree);
    claim(Si,Niagree);
    claim(Si,Nisynch);
  }

  role Pj
  {
    fresh Beta,Beta';var Alpha,Alpha',SKeyji;
    fresh TS2:Timestamp;var TS1:Timestamp;
    var pk1,add,mul: Function;
    macro SKSi = sk(Si);
    macro SKPj = sk(Pj);
    macro PKSi = pk1(SKSi,g);
    macro PKPj = pk1(SKpj,g);
    macro TRsi = mul(Alpha,g);
    macro TRpj = mul(Beta,g);
    macro h1 = H2(AIDSi,AIDPj,TRsi,PKSi,TS1);
    macro h1' = H2(AIDSi,AIDPj,TRsi,PKSi,TS1);
    macro C1 = {(h1)add(SKSi,Alpha)}Alpha';
    macro C2 = {(h1')add(SKpj,Beta)}Beta';
    macro TSKSi = mul(Alpha,TRpj);
    macro TSKPj = mul(TRsi,Beta);
    macro SKeyS = H3(AIDSi,AIDPj,TSKSi);
    macro SKeyP = H3(AIDSi,AIDPj,TSKPj);
    macro Mij = H4(SKeyS,TRsi,TS1,TS2);
    macro Mji = H4(SKeyP,TRsi,TS2,TS1);
    recv_1(Si,Pj,C1,TRsi,TS1);
    send_2(Pj,Si,C2,TRpj,TS2);
    recv_3(Si,Pj,Mij);
    send_4(Pj,Si,Mji);
    match (SKeyji,SKeyP);
    claim(Pj,Secret,Beta);
    claim(Pj,Secret,SKPj);
    claim(Pj,SKR,SKeyji);
    claim(Pj,Alive);
    claim(Pj,Weakagree);
    claim(Pj,Niagree);
    claim(Pj,Nisynch);
  }
}

```

The verification result of the AKA phase is shown in Figure 8. Scyther did not detect any methods that could be used for attacks. Specifically, the random numbers Alpha and Beta, the private keys $sk(S_i)$ and $sk(P_j)$ of both parties, and the session keys $SKey_{ij}$ and $SKey_{ji}$ were proven to be secure. Additionally, the verification results of the Alive, Weakagree, Niagree, and Nisynch properties for both entities were proven to be secure. Therefore, it demonstrates the scheme's security during the AKA phase.

5.3 Security properties analysis

In this section, we compare the proposed scheme with cross-domain authentication schemes (Shen et al., 2020; Dong et al., 2024; Gong et al., 2023; Xue et al., 2022) in terms of security properties, as shown in Table 4. For convenience, in the subsequent analysis, we refer to schemes (Shen et al., 2020; Dong et al., 2024; Gong et al., 2023; Xue et al., 2022) as BASA, BCFCa, LCDMA, and CBRCA, respectively. The selected schemes are all validated and representative of the current cross-domain authentication field. BASA, BCFCa, and CBRCA all integrate blockchain technology, while LCDMA implements lightweight cross-domain mutual identity authentication. BCFCa effectively addresses key escrow issues and CBRCA can support identity revocation. However, none of these existing solutions can simultaneously satisfy all essential security property requirements in Table 4. Following the comparison, we conduct a detailed analysis of the proposed scheme's security properties.

Figure 8 Verification results of the AKA phase (see online version for colours)

Scyther results : verify

Claim				Status	Comments
SWIM	Si	SWIM,Si1	Secret Alpha	Ok	No attacks within bounds.
		SWIM,Si2	Secret sk(Si)	Ok	No attacks within bounds.
		SWIM,Si3	SKR SKeyij	Ok	No attacks within bounds.
		SWIM,Si4	Alive	Ok	No attacks within bounds.
		SWIM,Si5	Weakagree	Ok	No attacks within bounds.
		SWIM,Si6	Niagree	Ok	No attacks within bounds.
		SWIM,Si7	Nisynch	Ok	No attacks within bounds.
Pj	SWIM,Pj1	Secret Beta	Ok	No attacks within bounds.	
		Secret sk(Pj)	Ok	No attacks within bounds.	
		SKR SKeyji	Ok	No attacks within bounds.	
		SWIM,Pj4	Alive	Ok	No attacks within bounds.
		SWIM,Pj5	Weakagree	Ok	No attacks within bounds.
		SWIM,Pj6	Niagree	Ok	No attacks within bounds.
		SWIM,Pj7	Nisynch	Ok	No attacks within bounds.

Done.

Table 4 Security properties comparison

<i>Security properties</i>	<i>BASA</i>	<i>LCDMA</i>	<i>CBRCA</i>	<i>BCFCA</i>	<i>Ours</i>
Forward security	√	√	√	√	√
Eavesdropping attack resistance	×	√	√	√	√
Identity impersonation attack resistance	√	√	√	√	√
Man-in-the-middle attack resistance	×	√	√	√	√
Replay attack resistance	×	√	√	√	√
DDoS attack resistance	√	√	√	√	√
Key escrow resistance	×	×	×	√	√
Anonymity	√	√	√	×	√
Mutual authentication	√	√	√	√	√
Revocable	×	×	√	×	√

- 1 Forward security: during each authentication interaction, the data publisher and data subscriber generate new random numbers α and β , respectively, to generate a shared session intermediate key TSK_{Si}/TSK_{Pj} . Subsequently, both parties use the session intermediate key to generate the session keys $SKey_{ij}/SKey_{ji}$ through hashing. Even if an attacker obtains the SK_{Si}/SK_{Pj} of the authenticated parties, the high freshness of the random numbers ensures that past session keys remain secure and the existing and sent session messages cannot be compromised. Therefore, the scheme provides forward security.
- 2 Eavesdropping attack resistance: the SK_{Si}/SK_{Pj} and the secret random numbers used during the registration and authentication processes are private to each entity. Upon receiving an authentication request, the recipient verifies the message's integrity and correctness. For instance, since the registration process uses an insecure channel, the registration requester calculates the hash value h_U and verifies the correctness and validity of SK_{Si}/SK_{Pj} using the PK_{CA} after receiving the intermediate key d_U and R_{CA} from the CA. Attackers cannot obtain valid confidential data by intercepting or eavesdropping on communications over insecure channels. Therefore, the scheme can resist eavesdropping attacks.
- 3 Identity impersonation attack resistance: users utilise AID_U instead of their real identity to complete the registration and AKA processes. Legitimate users' anonymous identities and public key information are stored and distributed through the blockchain ledger. Before initiating an authentication request, both parties can retrieve each other's public key and anonymous identity through smart contracts. The recipient of the authentication request uses their own private key and the sender's public key to verify the reliability of the authentication message. Since malicious attackers cannot generate the legitimate AID_U and obtain the sender's private key, the recipient cannot verify the request messages they send, resulting in an authentication interruption. Therefore, the scheme can effectively resist impersonation attacks.

- 4 Man-in-the-middle attack resistance: suppose an adversary Adv attempts a man-in-the-middle attack by intercepting the message $\{C_1, TS_1, TR_U\}$ from an insecure channel and tries to send a forged message $\{C_{1_Adv}, TS_{1_Adv}, TR_{U_Adv}\}$ to the recipient to spoof the authentication request. However, upon receiving the authentication request, the recipient uses the private key and the sender's public key to verify the correctness of C_1 , TS_1 and TR_U , thereby confirming whether the requester is genuine. In the authentication process, the data publisher and the data subscriber assume the authentication requestor or receiver roles, respectively. Both parties are required to encrypt or authenticate the message using their own private key and the message recipient's public key. This ensures the scheme can prevent man-in-the-middle attacks.
- 5 Replay attack resistance: during mutual authentication, the data publisher and data subscriber use timestamps to maintain freshness and use their respective private random values α and β to generate TR_U . Even if an adversary Adv intercepts and tampers with the message or inserts new timestamps to launch a replay attack, the high freshness of the timestamps and the avalanche effect of the hash function will cause the recipient's message integrity verification to fail. Therefore, the scheme can resist replay attacks.
- 6 DDoS attack resistance: the scheme introduces blockchain technology to store identity information and distribute keys for legitimate users. The decentralised and distributed storage characteristics of blockchain ensure the immutability and high availability of user identity and key data. Additionally, the AKA processes do not require the participation of a centralised entity. As long as the number of effective nodes supports blockchain consensus, the AKA processes will not be affected even if a single node or a domain's CA suffers a DDoS attack. Therefore, the scheme can effectively solve the single point of failure issue caused by DDoS attacks.
- 7 Key escrow resistance: unlike traditional centralised key generation schemes, the proposed scheme uses the CA to generate partial keys for registered users, addressing the key escrow problem. The CA does not generate the user's complete private key but only produces an intermediate key d_U for the registered user. The user then generates their private key using their own secret random number, AID_U , and the intermediate key provided by the CA. Since only the registered user has the complete private key, the scheme avoids the key escrow problem associated with centralised entities during user registration.
- 8 Identity anonymity: the user hashes his ID_U with a random number to generate AID_U , then sends AID_U to the CA to generate partial keys. Attackers cannot obtain the user's real identity information or deduce its real identity from the intercepted AID_U in the messages. Therefore, the scheme ensures the anonymity of the user's identity.

6 Performance analysis

In this section, we evaluate the performance of the scheme in terms of computational and communication overhead. Additionally, we compare it with existing cross-domain authentication schemes.

6.1 Experimental environment

The experimental environment for the scheme is set up with the following parameters: the 254-bit Barreto-Naehrig curve BN254 is defined as $E(F_p): y^2 = x^3 + 3$. It is an elliptic curve defined over a prime field F_p , where p is a 254-bit large prime number. We select two prime-order subgroups G_1 and G_2 on BN254 and set the bilinear pairing to $e: G_1 \times G_2 \rightarrow G_T$, where G_T is the target group. The secp256k1 elliptic curve is used for key agreement in some schemes. The symmetric encryption algorithm is the Advanced Encryption Standard (AES), utilising AES-256 in CBC mode for symmetric encryption. Regarding the experimental environment, we implemented various cryptographic operations using the Python Charm cryptographic library. The experiments were conducted in a virtual environment using VMware Workstation 17 Pro, running Ubuntu 22.04 LTS as the operating system. The virtual environment configuration included a quad-core virtual CPU (vCPU) and 8 GB of memory. The virtual machine ran on a Dell Precision 7540 with an Intel Core i7-9850H CPU @ 2.60 GHz processor. Regarding blockchain parameter settings, we choose Hyperledger Fabric v2.2.4 as a blockchain experimental platform and utilise raft as the consensus mechanism. Additionally, we use Hyperledger Caliper to simulate concurrent transaction requests and set the number of workers to 2.

6.2 Computational overhead

To accurately measure the computational overhead of each scheme during the registration and AKA phases, we have defined the basic cryptographic operations used in the schemes. The execution times of these cryptographic operations are presented in Table 5.

Table 5 The execution time of a cryptographic operation

<i>Symbols</i>	<i>Descriptions</i>	<i>Execution time (Ms)</i>
T_{ecm}	The execution time of point multiplication operations related to bilinear pairing	0.7862
T_{eca}	The execution time of point addition operations related to bilinear pairing	0.0033
T_{exp}	The execution time of a modular exponentiation operation in bilinear pairing.	1.9267
T_p	The execution time of a bilinear pairing operation.	20.1788
T_{ecm}^{ecc}	The execution time of a point multiplication operation on the secp256k1 curve	0.3426
T_{sen}	The execution time of a symmetric encryption operation	0.0532
T_h	The execution time of a hash operation	0.0032

In our scheme, the overall computational overhead of cryptographic operations is $18T_{ecm} + 12T_h + 6T_{eca}$. The computational overhead during the registration phase is slightly higher than that during the cross-domain AKA phase. This is because both entities involved in the authentication process need to register with the CA to obtain partial keys and compute their own private key SK_U and public key PK_U . We have also conducted a statistical and comparative analysis of the cryptographic operation overhead

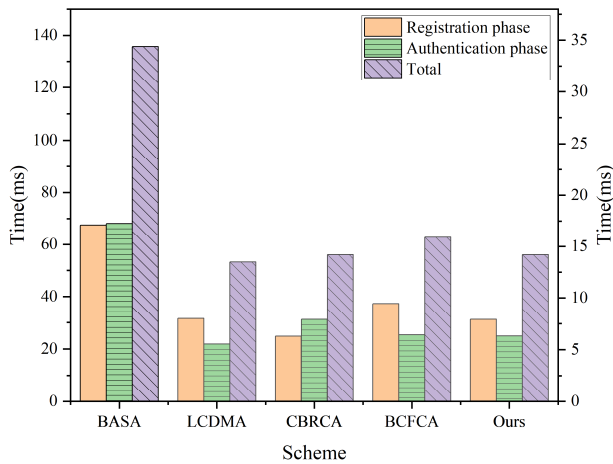
in the registration and AKA phases of the scheme and four existing cross-domain authentication schemes, as shown in Table 6.

Table 6 Comparison of computational overhead

<i>Scheme</i>	<i>Registration phase</i>	<i>AKA phase</i>	<i>Total</i>
BASA	$3T_p + 4T_{ecm} + 4T_h + 2T_{exp} + T_{eca}$	$3T_p + 3T_{ecm} + 4T_h + 2T_{exp} + T_{eca} + 4T_{ecm}^{ecc}$	$6T_p + 7T_{ecm} + 8T_h + 4T_{exp} + 2T_{eca} + 4T_{ecm}^{ecc}$
LCDMA	$10T_{ecm} + 9T_h + 2T_{sen}$	$7T_{ecm} + 10T_h$	$17T_{ecm} + 19T_h + 2T_{sen}$
CBRCA	$8T_{ecm} + 2T_h + 2T_{eca}$	$10T_{ecm} + 8T_h + 5T_{eca}$	$18T_{ecm} + 10T_h + 7T_{eca}$
BCFCA	$12T_{ecm} + 4T_{eca} + 2T_h$	$8T_{ecm} + 4T_h + 2T_{sen} + 2T_{eca}$	$20T_{ecm} + 6T_h + 2T_{sen} + 6T_{eca}$
Ours	$10T_{ecm} + 8T_h + 2T_{eca}$	$8T_{ecm} + 4T_h + 4T_{eca}$	$18T_{ecm} + 12T_h + 6T_{eca}$

The comparison results of computational costs are shown in Figure 9. For better visualisation, the computational overhead of the BASA scheme is represented using the left Y-axis scale, while the computational overhead of the other schemes is represented using the right Y-axis scale. In our LBCDA scheme, the total computational overhead for the mutual authentication process is approximately 14.2 ms. In contrast, the total execution time for one round of mutual authentication in the BASA, LCDMA, CBRCA, and BCFCA schemes is approximately 135.7 ms, 13.5 ms, 14.2 ms, and 15.9 ms, respectively. The BASA scheme has high computational overhead due to the extensive use of time-consuming bilinear pairing operations. In the comparative analysis of computational overhead, it can be observed that our scheme has a computational overhead similar to that of the CBRCA scheme. However, compared to the BASA and BCFCA schemes, our scheme has a lower computational cost. Although our scheme's computational overhead is slightly higher than that of the LCDMA scheme, we provide mutual confirmation of the session key, as opposed to LCDMA's unidirectional session key confirmation. Additionally, our scheme supports more comprehensive security properties compared to the other four related authentication schemes.

Figure 9 Bar chart of computational overhead comparison (see online version for colours)



6.3 Communication overhead

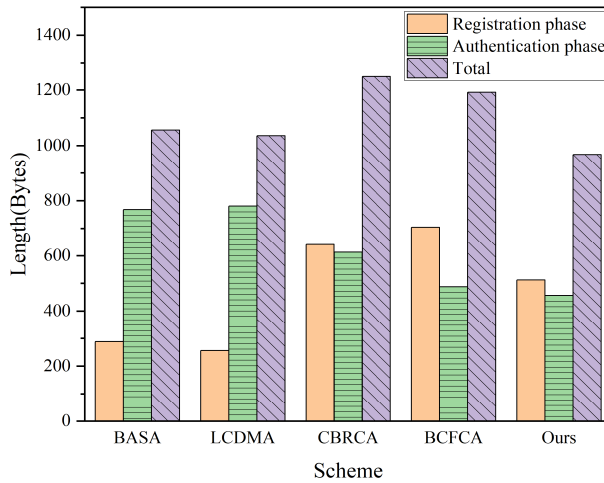
In this section, to uniformly analyse the performance of each scheme in terms of communication overhead, we made the following assumptions: the lengths of unique ID, AID, request message, and response message are all set to 32 bytes, denoted as $L_{|INFO|}$; the output of the hash function is 32 bytes, denoted as $L_{|H|}$; each coordinate is set to 254 bits, approximately 32 bytes, so the length of an element in the elliptic curve group is approximately 64 bytes, denoted as $L_{|G|}$; We set the input data length of the symmetric encryption algorithm to 32 bytes uniformly, then the output of symmetric encryption operations is 32 bytes, denoted as $L_{|C|}$; the size of an element in Z_p^* is 32 bytes, denoted as $L_{|NUM|}$; and the length of the timestamp is set to 4 bytes, denoted as $L_{|T|}$. The comparison results of communication overhead are shown in Table 7.

The comparison of communication overhead for each scheme is shown in Figure 10.

Table 7 Comparison of communication overhead

<i>Scheme</i>	<i>Registration phase</i>	<i>AKA phase</i>	<i>Total</i>
BASA	$4L_{ INFO } + L_{ H } + 2L_{ G }$	$11L_{ INFO } + 3L_{ H } + 4L_{ G } + 2L_{ NUM }$	$15L_{ INFO } + 4L_{ H } + 2L_{ NUM } + 6L_{ G }$
LCDMA	$6L_{ INFO } + 2L_{ NUM }$	$4L_{ INFO } + 6L_{ H } + 7L_{ G } + 3L_{ T }$	$10L_{ INFO } + 6L_{ H } + 2L_{ NUM } + 7L_{ G } + 3L_{ T }$
CBRCA	$4L_{ INFO } + 2L_{ H } + 2L_{ NUM } + 6L_{ G }$	$6L_{ INFO } + 2L_{ H } + L_{ NUM } + 5L_{ G } + L_{ T }$	$10L_{ INFO } + 4L_{ H } + 3L_{ NUM } + 11L_{ G } + L_{ T }$
BCFCA	$4L_{ INFO } + 2L_{ NUM } + 8L_{ G }$	$2L_{ INFO } + 4L_{ NUM } + 4L_{ G } + 2L_{ T } + L_{ C }$	$6L_{ INFO } + 6L_{ NUM } + 12L_{ G } + 2L_{ T } + L_{ C }$
Ours	$4L_{ INFO } + 4L_{ NUM } + 4L_{ G }$	$2L_{ INFO } + 2L_{ H } + 2L_{ NUM } + 4L_{ G } + 2L_{ T }$	$6L_{ INFO } + 2L_{ H } + 6L_{ NUM } + 8L_{ G } + 2L_{ T }$

Figure 10 Bar chart of communication overhead comparison (see online version for colours)



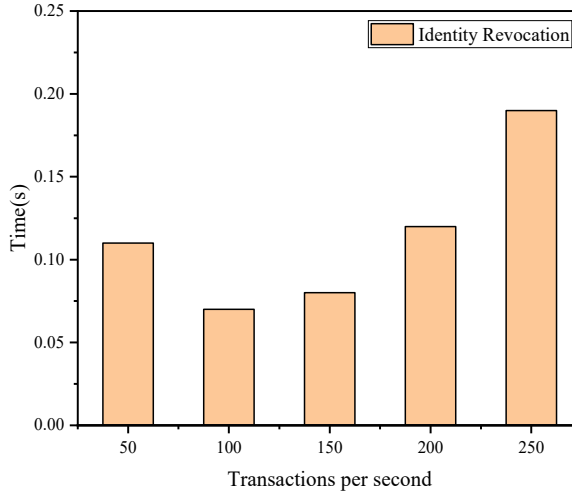
In our scheme, each user U needs to send data $\{AID_U, n\}$ and $\{AID_U, PK_U\}$, totalling $2L_{|INFO|} + L_{|NUM|} + L_{|G|} = 160$ bytes during the registration phase. The CA needs to return data $\{d_U, R_{CA}\}$ to U during the registration phase, corresponding to a data length of $L_{|NUM|}$.

$+ L_{|G|} = 96$ bytes. Since both entities S_i and P_j must complete the registration phase before entering the AKA phase. The total communication overhead during the registration phase is $4L_{|INFO|} + 4L_{|NUM|} + 4L_{|G|} = 512$ bytes. During the AKA phase, S_i needs to send data $\{C_1, TR_{S_i}, TS_1\}$ and M_{ij} and also needs to obtain P_j identity information $\{AID_{P_j}, PK_{P_j}\}$ returned from the SWIM-Chain, totalling $L_{|INFO|} + L_{|H|} + L_{|NUM|} + L_{|G|} + L_{|T|} = 228$ bytes. Similarly, P_j also needs to perform the same operations as S_i during the AKA phase. Therefore, the communication overhead for the proposed scheme during the AKA phase is $2 \times 228 = 456$ bytes. As shown in Table 7 and Figure 10, compared to the other four schemes, our scheme has a lower communication overhead, with a total communication overhead of approximately $6L_{|INFO|} + 2L_{|H|} + 6L_{|NUM|} + 8L_{|G|} + 2L_{|T|} = 968$ bytes.

6.4 Revocation latency

In this section, we conducted experiments to evaluate the average latency of the user identity revocation operation under varying transactions per second (TPS) conditions, with a fixed total transaction volume of 1,000. As illustrated in Figure 11, the identity revocation contract exhibits optimal performance at 100 TPS when utilising the Raft consensus algorithm. Beyond 100 TPS, the average latency shows a gradual increase. However, it remains consistently below 0.2 seconds even at 250 TPS. These results indicate that the revocation operation can maintain low latency under high-load scenarios, demonstrating the scheme's high efficiency and suitability for real-world applications under different volumes of revocation requests.

Figure 11 Average latency of identity revocation (see online version for colours)



7 Conclusions

The blockchain-assisted lightweight cross-domain authentication mechanism with privacy protection for SWIM proposed in this paper provides an effective scheme to solve the cross-domain authentication problem of lightweight users such as aircraft.

Through the combination of consortium blockchain and ECDH, secure and efficient cross-domain authentication and session key agreement are realised. Formal verification results show that the proposed scheme can achieve the expected authentication goal and is highly secure and reliable. However, the transaction processing capability and deployment environment of the existing blockchain architectures impose objective constraints on the authentication efficiency of the scheme, especially in high-concurrency scenarios. To address these limitations, our future research will focus on improving the node verification process of the consensus mechanism to increase the transaction processing throughput and introducing more performance metrics under realistic conditions to enhance the practicality.

Acknowledgements

This research was funded by the Education Commission Scientific Research Project of Tianjin (2022KJ081).

References

- Ahmad, R.W., Salah, K., Jayaraman, R., Hasan, H.R., Yaqoob, I. and Omar, M. (2021) ‘The role of blockchain technology in aviation industry’, *IEEE Aerospace and Electronic Systems Magazine*, March, Vol. 36, No. 3, pp.4–15, DOI: 10.1109/MAES.2020.3043152.
- Al-Riyami, S.S. and Paterson, K.G. (2003) ‘Certificateless public key cryptography’, in *Advances in Cryptology-ASIACRYPT 2003*, pp.452–473, Springer, Berlin, Germany, DOI: 10.1007/978-3-540-40061-5_29.
- Chen, J., Zhan, Z., He, K., Du, R., Wang, D. and Liu, F. (2022) ‘XAuth: efficient privacy-preserving cross-domain authentication’, *IEEE Transactions on Dependable and Secure Computing*, September, Vol. 19, No. 5, pp.3301–3311, DOI: 10.1109/TDSC.2021.3092375.
- Crescenzo, D.D., Strano, A. and Trausmuth, G. (2010) ‘SWIM: a next generation ATM information bus-the SWIM-SUIT prototype’, in *2010 14th IEEE International Enterprise Distributed Object Computing Conference Workshops*, IEEE, Vitria, Brazil, October, pp.41–46, DOI: 10.1109/EDOCW.2010.36.
- Cui, J., Liu, N., Zhang, Q., He, D., Gu, C. and Zhong, H. (2023) ‘Efficient and anonymous cross-domain authentication for IIoT based on blockchain’, *IEEE Transactions on Network Science and Engineering*, March-April, Vol. 10, No. 2, pp.899–910, DOI: 10.1109/TNSE.2022.3224453.
- Das, S. and Namasudra, S. (2023) ‘Lightweight and efficient privacy-preserving mutual authentication scheme to secure internet of things-based smart healthcare’, *Transactions on Emerging Telecommunications Technologies*, November, Vol. 34, No. 11, p.e4716, DOI: 10.1002/ett.4716.
- Dolev, D. and Yao, A. (1983) ‘On the security of public key protocols’, *IEEE Transactions on Information Theory*, March, Vol. 29, No. 2, pp.198–208, DOI: 10.1109/TIT.1983.1056650.
- Dong, J., Xu, G., Ma, C., Liu, J. and Cliff, U.G.O. (2024) ‘Blockchain-based certificate-free cross-domain authentication mechanism for industrial internet’, *IEEE Internet of Things Journal*, January, Vol. 11, No. 2, pp.3316–3330, DOI: 10.1109/JIOT.2023.3296506.
- Gong, B., Zheng, G., Waqas, M., Tu, S. and Chen, S. (2023) ‘LCDMA: lightweight cross-domain mutual identity authentication scheme for internet of things’, *IEEE Internet of Things Journal*, July, Vol. 10, No. 14, pp.12590–12602, DOI: 10.1109/JIOT.2023.3252051.

- He, D., Cai, Y., Zhu, S., Zhao, Z., Chan, S. and Guizani, M. (2023) 'A lightweight authentication and key exchange protocol with anonymity for IoT', *IEEE Transactions on Wireless Communications*, November, Vol. 22, No. 11, pp.7862–7872, DOI: 10.1109/TWC.2023.3257028.
- Huang, C. et al. (2022) 'Blockchain-assisted transparent cross-domain authorization and authentication for smart city', *IEEE Internet of Things Journal*, September, Vol. 9, No. 18, pp.17194–17209, DOI: 10.1109/JIOT.2022.3154632.
- Jia, X. et al. (2020) 'IRBA: an identity-based cross-domain authentication scheme for the internet of things', *Electronics*, April, Vol. 9, No. 4, p.634, DOI: 10.3390/electronics9040634.
- Klein, R., Abraham, B., Morales, A., Niewoehner, K., Aknine, F. and Pace, J. (2015) 'Aircraft access to SWIM – a collaborative ATM pathfinder to support a TBO environment', in *2015 Integrated Communication, Navigation and Surveillance Conference (ICNS)*, IEEE, Herndon, VA, USA, April, pp.Q4-1–Q4-10, DOI: 10.1109/ICNSURV.2015.7121256.
- Liu, Y. et al. (2024) 'Blockchain-based cross-domain authentication management system for IoT devices', *IEEE Transactions on Network Science and Engineering*, January-February, Vol. 11, No. 1, pp.115-127, DOI: 10.1109/TNSE.2023.3292624.
- Liu, Z. (2020) 'In-domain user identity authentication method for system wide information management (SWIM)', in *Proceedings of the 2020 12th International Conference on Machine Learning and Computing*, ACM, Shenzhen China, February, pp.479–482, DOI: 10.1145/3383972.3383981.
- Lu, X., Wu, Z. and Yue, M. (2024) 'T-ATMChain: blockchain-based identity authentication for air traffic management', *China Communications*, December, Vol. 21, No. 12, pp.186–202, DOI: 10.23919/jcc.ea.2021-0718.202401.
- Mall, P., Amin, R., Obaidat, M.S. and Hsiao, K-F. (2021) 'CoMSeC++: PUF-based secured lightweight mutual authentication protocol for drone-enabled WSN', *Computer Networks*, November, Vol. 199, p.108476, DOI: 10.1016/j.comnet.2021.108476.
- Qi, M. and Lu, S. (2017) 'Overview of system wide information management and security analysis', in *2017 IEEE 13th International Symposium on Autonomous Decentralized System (ISADS)*, Bangkok, IEEE, Thailand, March, pp.191–194, DOI: 10.1109/ISADS.2017.39.
- Shen, M., Liu, H., Zhu, L., Xu, K., Yu, H., Du, X. and Guizani, M. (2020) 'Blockchain-assisted secure device authentication for cross-domain industrial IoT', *IEEE Journal on Selected Areas in Communications*, May, Vol. 38, No. 5, pp.942–954, DOI: 10.1109/JSAC.2020.2980916.
- Song, R., Xiao, B., Song, Y., Guo, S. and Yang, Y. (2023) 'A survey of blockchain-based schemes for data sharing and exchange', *IEEE Transactions on Big Data*, December, Vol. 9, No. 6, pp.1477–1495, DOI: 10.1109/TBDATA.2023.3293279.
- Syverson, P.F. and van Oorschot, P.C. (1996) *A Unified Cryptographic Protocol Logic*, Naval Research Lab, Washington, DC, USA, Rep. ADA464967.
- Tang, Z. and Li, X. (2010) 'The formalization description of the Dolev-Yao intruder model', *Computer Engineering & Science*, Vol. 32, No. 8, pp.36–45.
- Wang, M., Rui, L., Yang, Y., Gao, Z. and Chen, X. (2022) 'A blockchain-based multi-CA cross-domain authentication scheme in decentralized autonomous network', *IEEE Transactions on Network and Service Management*, September, Vol. 19, No. 3, pp.2664–2676, DOI: 10.1109/TNSM.2022.3180357.
- Wu, Z., Zhao, T. and Lei, J. (2014) 'SWIM authentication method based on improved Diameter/EAP-MD5', *Journal on Communications*, August, Vol. 35, No. 8, pp.1–7, DOI: 10.3969/j.issn.1000-436x.2014.08.001.
- Xue, L., Huang, H., Xiao, F. and Wang, W. (2022) 'A cross-domain authentication scheme based on cooperative blockchains functioning with revocation for medical consortiums', *IEEE Transactions on Network and Service Management*, September, Vol. 19, No. 3, pp.2409–2420, DOI: 10.1109/TNSM.2022.3146929.

- Yang, X. et al. (2022) 'Blockchain-based secure and lightweight authentication for internet of things', *IEEE Internet of Things Journal*, March, Vol. 9, No. 5, pp.3321–3332, DOI: 10.1109/JIOT.2021.3098007.
- Yang, Y., Wei, L., Wu, J., Long, C. and Li, B. (2022) 'A blockchain-based multidomain authentication scheme for conditional privacy preserving in vehicular ad-hoc network', *IEEE Internet of Things Journal*, June, Vol. 9, No. 11, pp.8078–8090, DOI: 10.1109/JIOT.2021.3107443.
- Zhang, L., Bai, Z., Wu, Z. and Wang, K. (2023a) 'Unified identity authentication scheme of system wide information management based on SAML-PKI-LDAP', *International Journal of Information and Computer Security*, Vol. 22, No. 2, pp.186–209, DOI: 10.1504/IJICS.2023.134959.
- Zhang, L., Huang, Y., Nie, J. and Wang, K. (2023b) 'Cross-domain authentication scheme based on blockchain and consistent hash algorithm for system-wide information management', *Computers, Materials & Continua*, Vol. 77, No. 2, pp.1467–1488, DOI: 10.32604/cmc.2023.042305.
- Zhang, G., Hu, Q., Zhang, Y., Dai, Y. and Jiang, T. (2024a) 'Lightweight cross-domain authentication scheme for securing wireless IoT devices using backscatter communication', *IEEE Internet of Things Journal*, June, Vol. 11, No. 12, pp.22021–22035, DOI: 10.1109/JIOT.2024.3378476.
- Zhang, S., Yan, Z., Liang, W., Li, K-C. and Di Martino, B. (2024b) 'BCAE: a blockchain-based cross domain authentication scheme for edge computing', *IEEE Internet of Things Journal*, July, Vol. 11, No. 13, pp.24035–24048, DOI: 10.1109/JIOT.2024.3387934.
- Zou, W. et al. (2021) 'Smart contract development: challenges and opportunities', *IEEE Transactions on Software Engineering*, October, Vol. 47, No. 10, pp.2084–2106, DOI: 10.1109/TSE.2019.2942301.